

ML NOTES — বাংলা



Data Preprocessing Techniques

সম্পূর্ণ বাংলা গাইড — Missing Values, Encoding & Feature Scaling

Pandas

Scikit-learn

Imputation

One-Hot Encoding

Standardization

Normalization

📅 ২০২৬-০৪-০৪ | 🧑 Beginner to Intermediate



Data Preprocessing Techniques

Handling Missing Values | Encoding Categorical Data | Feature Scaling

বিষয়: Machine Learning Pipeline-এর সবচেয়ে গুরুত্বপূর্ণ ধাপ — Raw Data থেকে Clean Data তৈরি করা।

স্তর: Beginner to Intermediate | **ভাষা:** বাংলা (Technical terms ইংরেজিতে)

১. 🎯 সহজ ভাষায় পরিচিতি (Intuition)

এই concept কী এবং কেন দরকার?

কল্পনা করো তুমি একজন রাঁধুনি। তোমাকে বলা হলো একটি বিশেষ রেসিপি তৈরি করতে হবে। কিন্তু বাজার থেকে আনা সবজিগুলো এখনো কাঁচা, মাটি লাগা, কিছু পচা — এগুলো সরাসরি রান্নায় দেওয়া যাবে না। তোমাকে প্রথমে:

- পচা সবজি বাদ দিতে হবে বা ঠিক করতে হবে → Missing Values Handle করা
- সবজিগুলোকে নির্দিষ্ট আকারে কাটতে হবে → Encoding Categorical Data

• সব উপকরণ একই পরিমাণে মাপতে হবে → Feature Scaling

Machine Learning-এ Raw Data ঠিক এই কাঁচা সবজির মতো। Model সরাসরি এই Data দিয়ে ভালো কাজ করবে না। Data Preprocessing হলো সেই প্রক্রিয়া যা Raw Data-কে Model-এর জন্য উপযুক্ত করে তোলে।

বাস্তব জীবনের উদাহরণ:

স্কুলের পরীক্ষার রেজাল্ট শিট:

নাম	বাংলা	ইংরেজি	গণিত	শহর	ফলাফল
রাহিম	75	80	NaN	ঢাকা	পাস
করিম	60	NaN	90	চট্টগ্রাম	পাস
নাসরিন	NaN	70	85	ঢাকা	ফেল
তারেক	55	65	70	রাজশাহী	পাস

এই Table-এ ৩টি সমস্যা আছে:

- NaN (Missing Values): কিছু ঘর ফাঁকা — কম্পিউটার জানে না এখানে কী বসাবে
- শহর Column (Categorical Data): "ঢাকা", "চট্টগ্রাম" — কম্পিউটার এই text বুঝতে পারে না
- বিভিন্ন স্কেল নেই এখানে কিন্তু বাস্তবে বয়স (1-100) এবং বেতন (10,000-10,00,000) একসাথে দিলে সমস্যা হয়

এটি কোন সমস্যা সমাধান করে?

সমস্যা	সমাধান
ফাঁকা ডেটা থাকলে model crash করে	Missing Value Handling
Text data model বুঝতে পারে না	Encoding
বড় সংখ্যা ছোট সংখ্যাকে dominate করে	Feature Scaling
Gradient Descent ধীরে কাজ করে	Standardization/Normalization

২. 📖 বিস্তারিত ব্যাখ্যা (Deep Explanation)

২.১ — Missing Values Handle করা

Missing Value কী?

Dataset-এ যখন কোনো cell-এ কোনো মান নেই, সেটিকে Missing Value বলে। Python-এ এটি `NaN` (Not a Number), `None`, বা `null` হিসেবে দেখা যায়।

Missing Value কেন তৈরি হয়?

- **Data Collection সমস্যা:** Survey-এ কেউ সব প্রশ্নের উত্তর দেননি
- **System Error:** Database থেকে data export করার সময় error হয়েছে
- **Sensor Failure:** IoT device-এর sensor কিছু সময় কাজ করেনি
- **Privacy কারণ:** কেউ তার বেতন বা বয়স দিতে চাননি
- **Data Merging:** দুটি dataset একত্রিত করার সময় কিছু row-এ কলাম match করেনি

Missing Value-এর ৩ ধরন (MAR, MCAR, MNAR):

MCAR: Missing Completely At Random

→ যেকোনো কারণে random ভাবে missing

→ Example: Survey form-এ দুর্ঘটনাবশত ফাঁকা রেখে দেওয়া

MAR: Missing At Random

→ Missing হওয়াটা অন্য variable-এর উপর নির্ভর করে

→ Example: পুরুষরা বেতন বলতে চান না →

gender জানলে missing predict করা সম্ভব

MNAR: Missing Not At Random

→ Missing হওয়া নিজেই একটা pattern

→ Example: অসুস্থ রোগীরা checkup-এ আসেন না →

health data-তে healthy লোকের bias

Missing Value Handle করার পদ্ধতি:

পদ্ধতি ১: Deletion (মুছে ফেলা)

Listwise Deletion: পুরো row মুছে দেওয়া

আগে:		পরে:
রাহিম NaN		করিম 90
করিম 90	→	তারেক 70
নাসরিন NaN		(NaN row মুছা হলো)
তারেক 70		

- ✓ সহজ
- ✗ ডেটা কমে যায়, Bias তৈরি হতে পারে

Column Deletion: যদি কোনো column-এ ৭০%+ missing হয়, সেই column মুছে দাও

পদ্ধতি ২: Imputation (ভরাট করা)

Numerical Data-র জন্য:

পদ্ধতি	কখন ব্যবহার করব
Mean (গড়)	Data এ Outlier নেই, Normal distribution
Median (মধ্যক)	Outlier আছে (যেমন বেতন data)
Mode (সর্বোচ্চ বারবার আসা মান)	Categorical data
Forward Fill / Backward Fill	Time-series data (স্টক মার্কেট)
KNN Imputation	অন্য features দেখে predict করা
Constant (-999 বা "Unknown")	Missing নিজেই তথ্য বহন করে

২.২ — Encoding Categorical Data

Categorical Data কী?

যে Data সংখ্যার পরিবর্তে category বা text হিসেবে থাকে:

- Nominal (নামসূচক): কোনো ক্রম নেই — রং (লাল, নীল, সবুজ), শহর
- Ordinal (ক্রমসূচক): ক্রম আছে — শিক্ষার স্তর (SSC < HSC < BSc < MSc)

কেন Encoding দরকার?

Machine Learning algorithm গণিত করে। "ঢাকা" + "চট্টগ্রাম" বা "লাল" × 5 — এগুলো অর্থহীন। তাই text-কে number-এ রূপান্তর করতে হয়।

Encoding পদ্ধতিসমূহ:

পদ্ধতি ১: Label Encoding

প্রতিটি category-কে একটি সংখ্যা দেওয়া হয়।

শিক্ষার স্তর → Label Encoded

SSC → 0

HSC → 1

BSc → 2

MSc → 3

ব্যবহার: শুধুমাত্র Ordinal (ক্রমানুসারে) data-র জন্য।

সমস্যা: Nominal data-য় ব্যবহার করলে model ভুল বোঝে (ভাবে $MSc = 3 \times SSC$)

পদ্ধতি ২: One-Hot Encoding

প্রতিটি category-র জন্য নতুন binary column তৈরি হয়।

শহর column: One-Hot Encoded:

ঢাকা → is_dhaka=1, is_ctg=0, is_raj=0

চট্টগ্রাম → is_dhaka=0, is_ctg=1, is_raj=0

রাজশাহী → is_dhaka=0, is_ctg=0, is_raj=1

ব্যবহার: Nominal data-র জন্য আদর্শ।

সমস্যা: অনেক category থাকলে অনেক column তৈরি হয় → Curse of Dimensionality

Dummy Variable Trap এড়ানো:

যদি `is_dhaka=0` এবং `is_ctg=0` হয়, তাহলে আমরা বুঝতে পারি এটি রাজশাহী। তাই একটি column বাদ দেওয়া যায় (`drop_first=True`)।

পদ্ধতি ৩: Target (Mean) Encoding

প্রতিটি category-কে সেই category-র target variable-এর গড় দ্বারা replace করা হয়।

শহর | বেতন_গড়
ঢাকা | 50,000 → ঢাকা → 50000
চট্টগ্রাম | 40,000 → চট্টগ্রাম → 40000

ব্যবহার: অনেক বেশি category (high cardinality) থাকলে।

সতর্কতা: Data leakage-এর ঝুঁকি আছে।

২.৩ — Feature Scaling

Feature Scaling কী এবং কেন দরকার?

ধরো তুমি একজন ছাত্রের সাফল্য predict করতে চাও দুটি feature দিয়ে:

- **বয়স:** 15 থেকে 25 (small range)
- **পরিবারের বার্ষিক আয়:** 1,00,000 থেকে 50,00,000 (huge range)

আয়ের সংখ্যাগুলো এত বড় যে model শুধু আয়ের দিকে মনোযোগ দেবে, বয়সকে প্রায় ignore করবে। এটি ঠিক না! Feature Scaling এই সমস্যার সমাধান করে।

কোন অ্যালগরিদমে Scaling দরকার:

Scaling দরকার ✓

Scaling দরকার নেই ✗

KNN

Decision Tree

SVM

Random Forest

Scaling দরকার ✓

Scaling দরকার নেই ✗

Linear/Logistic Regression

Gradient Boosting (XGBoost)

Neural Networks

Naive Bayes

PCA

K-Means Clustering

দুটি প্রধান Scaling পদ্ধতি:

পদ্ধতি ১: Normalization (Min-Max Scaling)

Data-কে 0 থেকে 1 (বা -1 থেকে 1) এর মধ্যে নিয়ে আসে।

পদ্ধতি ২: Standardization (Z-Score Scaling)

Data-কে Mean=0, Standard Deviation=1 করে রূপান্তরিত করে।

৩. Math / Theory

৩.১ — Missing Value Stats

Missing Percentage বের করা:

$$\text{Missing \%} = (\text{Missing মানের সংখ্যা} / \text{মোট মানের সংখ্যা}) \times 100$$

Mean (গড়):

$$\text{Mean} = \frac{x_1 + x_2 + x_3 + \dots + x_n}{n}$$

উদাহরণ: [75, NaN, 60, 90, 70] → NaN বাদে Mean = (75+60+90+70)/4 = 73.75

Median (মধ্যক):

Sorted: [60, 70, 75, 90]

Median = (70 + 75) / 2 = 72.5 [জোড় সংখ্যার ক্ষেত্রে দুই মধ্যের গড়]

৩.২ — Normalization (Min-Max Scaling) সূত্র

$$X_{\text{normalized}} = \frac{X - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}}$$

প্রতিটি Symbol-এর অর্থ:

- X → Original মান যেটি scale করতে চাই
- X_{min} → ওই column-এর সর্বনিম্ন মান
- X_{max} → ওই column-এর সর্বোচ্চ মান
- $X_{\text{normalized}}$ → 0 থেকে 1 এর মধ্যে রূপান্তরিত মান

Manual Calculation:

বয়স data: [15, 20, 22, 25, 18]

- $X_{\text{min}} = 15, X_{\text{max}} = 25$

বয়স (X)	হিসাব	Normalized
15	$(15-15)/(25-15) = 0/10$	0.0
20	$(20-15)/(25-15) = 5/10$	0.5
22	$(22-15)/(25-15) = 7/10$	0.7
25	$(25-15)/(25-15) = 10/10$	1.0
18	$(18-15)/(25-15) = 3/10$	0.3

ফলাফল: [0.0, 0.5, 0.7, 1.0, 0.3] → সব মান 0 থেকে 1 এর মধ্যে

৩.৩ — Standardization (Z-Score) সূত্র

$$X_{\text{standardized}} = \frac{X - \mu}{\sigma}$$

প্রতিটি Symbol-এর অর্থ:

- X → Original মান
- μ (μ) → Column-এর Mean (গড়)
- σ (σ) → Column-এর Standard Deviation (মানক বিচ্যুতি)
- $X_{\text{standardized}}$ → Standardized মান (Z-score)

Standard Deviation বের করার সূত্র:

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$$

Manual Calculation:

বয়স data: [15, 20, 22, 25, 18]

ধাপ ১ — Mean বের করি:

$$\begin{aligned}\mu &= (15 + 20 + 22 + 25 + 18) / 5 \\ \mu &= 100 / 5 \\ \mu &= 20\end{aligned}$$

ধাপ ২ — Standard Deviation বের করি:

প্রতিটির জন্য $(x - \mu)^2$:

$$15: (15-20)^2 = (-5)^2 = 25$$

$$20: (20-20)^2 = (0)^2 = 0$$

$$22: (22-20)^2 = (2)^2 = 4$$

$$25: (25-20)^2 = (5)^2 = 25$$

$$18: (18-20)^2 = (-2)^2 = 4$$

$$\text{গড় variance} = (25+0+4+25+4)/5 = 58/5 = 11.6$$

$$\sigma = \sqrt{11.6} \approx 3.406$$

ধাপ ৩ — Z-score বের করি:

বয়স (X)	হিসাব	Z-Score
15	$(15-20)/3.406$	-1.468
20	$(20-20)/3.406$	0.000
22	$(22-20)/3.406$	+0.587
25	$(25-20)/3.406$	+1.468
18	$(18-20)/3.406$	-0.587

ফলাফল: [-1.468, 0.0, 0.587, 1.468, -0.587] → Mean=0, SD≈1

Z-Score এর অর্থ:

- $z = 0$ → এই মানটি ঠিক গড়ের সমান
- $z = +1$ → গড় থেকে 1 standard deviation উপরে
- $z = -1$ → গড় থেকে 1 standard deviation নিচে
- $|z| > 3$ → সম্ভাব্য Outlier!

৩.৪ — Normalization vs Standardization তুলনা

বৈশিষ্ট্য	Normalization	Standardization
সূত্র	$(X - \min) / (\max - \min)$	$(X - \mu) / \sigma$
Range	[0, 1] (bounded)	$[-\infty, +\infty]$ (unbounded)
Mean	পরিবর্তিত হয়	0 হয়
Outlier Effect	বেশি প্রভাবিত	কম প্রভাবিত
Distribution	বজায় থাকে	Normal-এর কাছাকাছি হয়
কখন ব্যবহার	Image pixels, Neural Networks	SVM, PCA, Linear Regression

৪. Code Example (Python)

```

# প্রয়োজনীয় library import করি
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, MinMaxScaler, StandardScaler

# =====
# ডেমো Dataset তৈরি করি (বাস্তব জীবনের মতো)
# =====
data = {
    'নাম': ['রাহিম', 'করিম', 'নাসরিন', 'তারেক', 'সুমা', 'জামাল'],
    'বয়স': [25, np.nan, 30, 22, np.nan, 28], # Missing আছে
    'বেতন': [50000, 40000, np.nan, 35000, 60000, 45000], # Missing আছে
    'শহর': ['ঢাকা', 'চট্টগ্রাম', 'ঢাকা', 'রাজশাহী', 'চট্টগ্রাম', 'ঢাকা'], # Categorical
    'শিক্ষা': ['MSc', 'BSc', 'HSC', 'MSc', 'PhD', 'BSc'], # Ordinal Categorical
    'পাস': [1, 1, 0, 1, 1, 0] # Target variable
}

df = pd.DataFrame(data)
print("=" * 60)
print("📄 মূল Dataset:")
print("=" * 60)
print(df.to_string())
print()

# =====
# ধাপ ১: Missing Values দেখি
# =====
print("=" * 60)
print("🔍 Missing Value Analysis:")
print("=" * 60)
print(f"মোট Missing Values:\n{df.isnull().sum()}")
print(f"\nMissing Percentage:")
missing_pct = (df.isnull().sum() / len(df)) * 100
print(missing_pct.round(2))
print()

# =====
# ধাপ ২: Missing Values Handle করা
# =====
print("=" * 60)
print("🛠️ Missing Values Handle করছি:")

```

```

print("=" * 60)

df_processed = df.copy() # Original data নষ্ট না করতে copy নিচ্ছি

# Numerical column-এ Median দিয়ে ভরাট (Outlier-safe)
# বয়সে Mean ব্যবহার করি (কারণ outlier নেই)
age_mean = df_processed['বয়স'].mean()
df_processed['বয়স'].fillna(age_mean, inplace=True)
print(f"✅ বয়স column: Mean={age_mean:.1f} দিয়ে NaN ভরাট করা হলো")

# বেতনে Median ব্যবহার করি (outlier থাকতে পারে)
salary_median = df_processed['বেতন'].median()
df_processed['বেতন'].fillna(salary_median, inplace=True)
print(f"✅ বেতন column: Median={salary_median:.0f} দিয়ে NaN ভরাট করা হলো")

print("\nMissing Values-এর পরে:")
print(df_processed[['বয়স', 'বেতন']].to_string())
print()

# =====
# ধাপ ৩: Categorical Data Encoding
# =====

print("=" * 60)
print("📦 Categorical Encoding:")
print("=" * 60)

## --- Label Encoding: শিক্ষা (Ordinal) ---
# ক্রম মেনে encoding করি: HSC < BSc < MSc < PhD
education_order = {'HSC': 0, 'BSc': 1, 'MSc': 2, 'PhD': 3}
df_processed['শিক্ষা_encoded'] = df_processed['শিক্ষা'].map(education_order)
print("✅ শিক্ষা (Ordinal Label Encoding):")
print(df_processed[['শিক্ষা', 'শিক্ষা_encoded']].to_string())
print()

## --- One-Hot Encoding: শহর (Nominal) ---
# drop_first=True → Dummy Variable Trap এড়ানোর জন্য
city_dummies = pd.get_dummies(
    df_processed['শহর'],
    prefix='শহর', # column নামের prefix
    drop_first=True # প্রথম column বাদ দিই (multicollinearity এড়াতে)
)
print("✅ শহর (One-Hot Encoding):")
print(city_dummies.to_string())
print()

```

```

# নতুন columns যোগ করি, পুরনো 'শহর' column মুছি
df_processed = pd.concat([df_processed, city_dummies], axis=1)
df_processed.drop(columns=['শহর', 'নাম'], inplace=True) # text column মুছি
print("Dataset (Encoding-এর পরে):")
print(df_processed.to_string())
print()

# =====
# ধাপ 8: Feature Scaling
# =====

print("=" * 60)
print("🔪 Feature Scaling:")
print("=" * 60)

# Scale করার জন্য numerical columns বেছে নিচ্ছি
numerical_cols = ['বয়স', 'বেতন']

# --- Normalization (Min-Max Scaling) ---
min_max_scaler = MinMaxScaler()
df_normalized = df_processed.copy()

# fit_transform: প্রথমে min/max শেখে, তারপর transform করে
df_normalized[numerical_cols] = min_max_scaler.fit_transform(
    df_normalized[numerical_cols]
)

print("✅ Normalization (Min-Max) - বয়স ও বেতন:")
print(df_normalized[numerical_cols].round(4).to_string())
print()

# --- Standardization (Z-Score) ---
std_scaler = StandardScaler()
df_standardized = df_processed.copy()

# fit_transform: প্রথমে mean/std শেখে, তারপর transform করে
df_standardized[numerical_cols] = std_scaler.fit_transform(
    df_standardized[numerical_cols]
)

print("✅ Standardization (Z-Score) - বয়স ও বেতন:")
print(df_standardized[numerical_cols].round(4).to_string())
print()

# --- তুলনা করি ---
print("=" * 60)

```

```
print("📊 Scaling তুলনা (বয়স column):")
print("=" * 60)

comparison = pd.DataFrame({
    'Original': df_processed['বয়স'].values,
    'Normalized (0-1)': df_normalized['বয়স'].values.round(4),
    'Standardized (Z)': df_standardized['বয়স'].values.round(4)
})

print(comparison.to_string())

# Statistics check
print("\n📊 Statistics after Standardization:")
print(f"Mean (০ হওয়া উচিত): {df_standardized['বয়স'].mean():.6f}")
print(f"Std Dev (1 হওয়া উচিত): {df_standardized['বয়স'].std():.6f}")
print(f"\nNormalized range: [{df_normalized['বয়স'].min():.2f},
{df_normalized['বয়স'].max():.2f}]"
```


Expected Output:

=====

 মূল Dataset:

=====

	নাম	বয়স	বেতন	শহর	শিক্ষা	পাস
0	রাহিম	25.0	50000.0	ঢাকা	MSc	1
1	করিম	NaN	40000.0	চট্টগ্রাম	BSc	1
2	নাসরিন	30.0	NaN	ঢাকা	HSC	0
3	তারেক	22.0	35000.0	রাজশাহী	MSc	1
4	সুমা	NaN	60000.0	চট্টগ্রাম	PhD	1
5	জামাল	28.0	45000.0	ঢাকা	BSc	0

=====

 Missing Value Analysis:

=====

মোট Missing Values:

বয়স 2

বেতন 1

...

 বয়স column: Mean=26.3 দিয়ে NaN ভরাট করা হলো বেতন column: Median=47500.0 দিয়ে NaN ভরাট করা হলো Scaling তুলনা (বয়স column):

	Original	Normalized (0-1)	Standardized (Z)
0	25.0	0.3500	-0.3244
1	26.3	0.5244	0.0000
2	30.0	1.0000	0.9388
3	22.0	0.0000	-1.1632
4	26.3	0.5244	0.0000
5	28.0	0.7000	0.4322

 Statistics after Standardization:

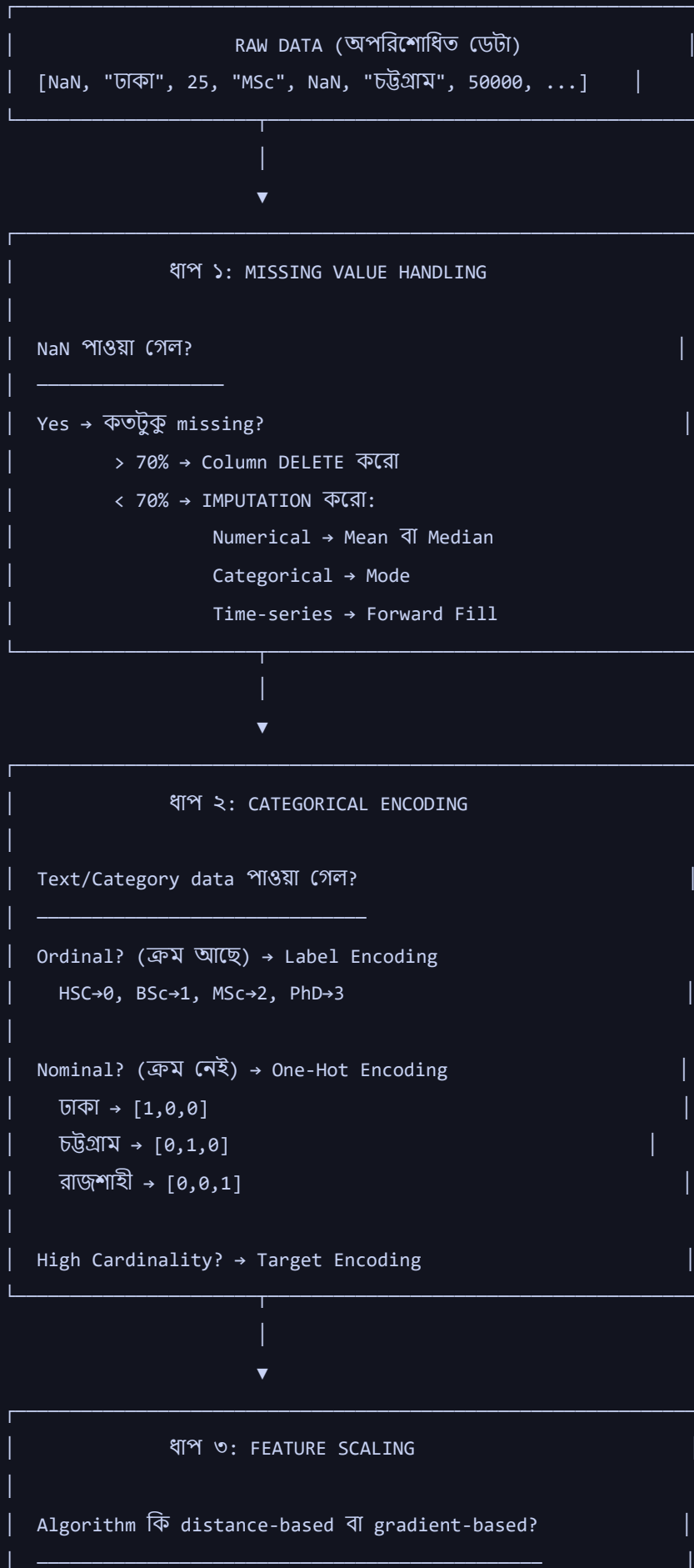
Mean (0 হওয়া উচিত): 0.000000

Std Dev (1 হওয়া উচিত): 1.000000

Normalized range: [0.00, 1.00]

৫. 🎨 Visual / Diagram

সম্পূর্ণ Data Preprocessing Pipeline:



Yes → Scale করতে হবে!

Data-তে Outlier আছে?

Outlier আছে → Standardization (Z-Score) ব্যবহার করো

formula: $Z = (X - \mu) / \sigma$

result: Mean=0, SD=1

Outlier নেই, bounded range চাই → Normalization

formula: $X_{\text{norm}} = (X - \min) / (\max - \min)$

result: সব মান [0, 1] এর মধ্যে



CLEAN, PROCESSED DATA ☒

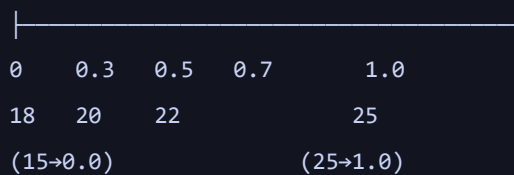
সব Numerical, কোনো NaN নেই, সব একই scale-এ

→ এখন ML Model Train করা যাবে!

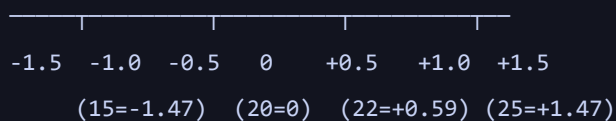
Normalization vs Standardization — Visual তুলনা:

Original Data: [15, 20, 22, 25, 18]

Normalization (0 to 1):



Standardization (Z-Score):



One-Hot Encoding Visual:

শহর column:

ঢাকা
চট্টগ্রাম
ঢাকা
রাজশাহী
চট্টগ্রাম

→

One-Hot Encoded:

is_dhaka	is_ctg	is_raj
1	0	0
0	1	0
1	0	0
0	0	1
0	1	0

৬. Real-world Use Cases

১. স্বাস্থ্যসেবা — রোগ পূর্বাভাস (Disease Prediction)

কোম্পানি: Apollo Hospitals, IBM Watson Health

সমস্যা: রোগীর রেকর্ডে অনেক missing lab values, ভিন্ন unit-এ পরিমাপ

সমাধান:

- Missing blood test results → KNN Imputation দিয়ে অনুমান করা
- রোগের নাম (ICD codes) → Label Encoding
- রক্তচাপ ও বয়স → Standardization (কারণ neural network ব্যবহার হয়)

২. E-commerce — পণ্য সুপারিশ (Product Recommendation)

কোম্পানি: Daraz, Amazon, Chaldal

সমস্যা: Customer rating data-তে অনেক missing, product category text

সমাধান:

- Missing product ratings → Median Imputation
- Product Category (Electronics, Clothing etc.) → One-Hot Encoding

- Price ও Ratings → Min-Max Normalization (KNN algorithm-এর জন্য)

৩. 🏠 ব্যাংকিং — Loan Default Prediction

কোম্পানি: BRAC Bank, Dutch-Bangla Bank

সমস্যা: কিছু আবেদনকারী সব তথ্য দেননি, employment type এবং district text

সমাধান:

- Missing income → Mode imputation (সবচেয়ে সাধারণ income bracket)
- Employment type (চাকরি/ব্যবসা/কৃষি) → One-Hot Encoding
- Income ও debt → Standardization (Logistic Regression-এর জন্য)

৪. 🌾 কৃষি — ফসলের পরিমাণ পূর্বাভাস

সংস্থা: FAO, Bangladesh Agricultural Research Institute

সমস্যা: Sensor malfunction-এ temperature data missing, soil type categorical

সমাধান:

- Missing temperature readings → Forward Fill (time-series)
- Soil type (এঁটেল/দোআঁশ/বেলো) → One-Hot Encoding
- Rainfall ও temperature → Normalization (Neural Network)

৫. 🎓 শিক্ষা — ছাত্রছাত্রীদের ফলাফল পূর্বাভাস

প্রতিষ্ঠান: Robi 10 Minute School, BUET HEQEP

সমস্যা: Attendance missing, course type categorical, marks বিভিন্ন scale-এ

সমাধান:

- Missing attendance → Mean Imputation
- Course type → Label Encoding (সহজ থেকে কঠিন)
- Marks (100-এ) ও CGPA (4-এ) → Normalization

৭. ⚖️ Pros & Cons

Missing Value Handling:

সুবিধা ✓

অসুবিধা ✗

Model crash-এর সম্ভাবনা কমায়

Mean imputation variance কমিয়ে দেয়

Data সম্পূর্ণ থাকে

Row deletion-এ $\frac{1}{2}$ তথ্য হারায়

Pattern learn করতে সাহায্য করে

ভুল imputation bias তৈরি করে

Downstream analysis সহজ হয়

MNAR ডেটা সঠিকভাবে handle করা কঠিন

Categorical Encoding:

সুবিধা ✓

অসুবিধা ✗

Numerical model ব্যবহার করা যায়

One-Hot Encoding-এ অনেক columns তৈরি হয়

Ordinal তথ্য সংরক্ষিত থাকে

Label Encoding Nominal data-য় ভুল order তৈরি করে

Model দ্রুত train হয়

Memory ব্যবহার বাড়তে পারে

Algorithm flexibility বাড়ে

High cardinality-তে inefficient

Feature Scaling:

সুবিধা ✓

অসুবিধা ✗

Model convergence দ্রুত হয়

Interpretability কমে যায়

Distance-based algo improve হয়

Outlier Normalization-কে distort করে

Gradient Descent smooth হয়

Scaler parameter save করতে হয়

সব feature সমান গুরুত্ব পায়

Tree-based model-এ অপ্রয়োজনীয়

৮. ⚠ Common Mistakes & Gotchas

ভুল ১: Test Data-তে fit করা (Data Leakage!)

```
# ❌ ভুল – Test data দেখে scaler train হচ্ছে
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.fit_transform(X_test) # ❌ fit_transform নয়!

# ✅ সঠিক – শুধু Train data দিয়ে fit করো
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # Train-এ fit ও transform
X_test_scaled = scaler.transform(X_test) # Test-এ শুধু transform!
```

কারণ: Real world-এ test data (ভবিষ্যৎ ডেটা) আগে থেকে জানা নেই। Scaler train data দেখে "শেখে", তারপর test data-কে সেই অনুযায়ী scale করে।

ভুল ২: Target Variable Scale করা

```
# ❌ ভুল – y (target) scale করবে না
scaler.fit_transform(X_train)
y_scaled = scaler.fit_transform(y_train) # ❌ এটি করবে না!

# ✅ শুধু feature (X) scale করো, target (y) নয়
```

ভুল ৩: Label Encoding Nominal Data-তে


```
# ❌ ভুল – শহর Nominal, তবুও Label Encoding
le = LabelEncoder()
df['শহর'] = le.fit_transform(df['শহর'])
# ঢাকা=0, চট্টগ্রাম=1, রাজশাহী=2 → Model ভাবে রাজশাহী > চট্টগ্রাম > ঢাকা! ❌

# ✅ Nominal data-য় One-Hot Encoding
pd.get_dummies(df['শহর'], drop_first=True)
```

ভুল 8: Imputation-এর আগে Train-Test Split না করা

```
# ❌ ভুল – পুরো dataset এ imputation করছে (data leakage)
df['বয়স'].fillna(df['বয়স'].mean(), inplace=True)
X_train, X_test = train_test_split(df)

# ✅ সঠিক – Split করার পরে imputation
X_train, X_test = train_test_split(df)
train_mean = X_train['বয়স'].mean() # শুধু train-এর mean
X_train['বয়স'].fillna(train_mean, inplace=True)
X_test['বয়স'].fillna(train_mean, inplace=True) # train-এর mean দিয়ে test fill করো
```

ভুল ৫: Tree-based Model-এ Scaling করা

```
# ⚠️ অপ্রয়োজনীয় – RandomForest-এ scaling কোনো পার্থক্য করে না
X_scaled = scaler.fit_transform(X)
model = RandomForestClassifier().fit(X_scaled, y) # scaling এখানে অর্থহীন

# ✅ Tree-based model-এ scale করার দরকার নেই
model = RandomForestClassifier().fit(X, y) # সরাসরি ব্যবহার করো
```

ভুল ৬: কোন Imputation সেটা না ভেবেই করা

- # ⚠ সতর্কতা: সবসময় mean imputation করা ঠিক না
- # বৈতনিক ডেটায় (outlier থাকে): Mean > Median ব্যবহারে bias
- # রোগীর বয়স (skewed): Median ব্যবহার করো
- # শহরের নাম: Mode (সবচেয়ে বেশি আসা শহর) ব্যবহার করো

৯. 🔗 Related Topics

আগে যা জানা দরকার (Prerequisites):

বিষয়	কারণ
Pandas & NumPy	DataFrame manipulation, NaN handle করা
Statistics (Basic)	Mean, Median, Mode, Standard Deviation বোঝা
Python Basics	Loop, Function, Library import
Exploratory Data Analysis (EDA)	Data-র pattern বোঝা, visualization

পরে যা শেখা উচিত (Next Steps):

বিষয়	সংযোগ
Feature Engineering	Preprocessing-এর পরের ধাপ — নতুন feature তৈরি
Linear Regression	Clean data দিয়ে প্রথম model
Logistic Regression	Classification-এ Standardization গুরুত্বপূর্ণ

বিষয়	সংযোগ
Pipeline (sklearn)	সব preprocessing steps একত্রিত করা
Cross Validation	Data leakage ছাড়া model evaluate করা
Outlier Detection	Preprocessing-এর আরেকটি গুরুত্বপূর্ণ অংশ

So. 🧠 Memory Tricks

মনে রাখার সহজ কৌশল:

Missing Values → "MOM" Strategy:

- More than 70% missing → Move on (column delete)
- Outlier আছে → Median ব্যবহার করো
- Missing কম → Mean/Mode দিয়ে ভরাট করো

Encoding → "NO" Rule:

- Nominal data → One-Hot Encoding
- Ordinal data → Label Encoding (Ordered) — মনে রাখো: "OL → Order Label"

Scaling → "Outlier decides":

- Outlier আছে → Standardization (Z-score) — "S for Safe from Outliers"
- Outlier নেই, bounded range চাই → Normalization — "N for Nice 0-1 range"

Data Leakage মনে রাখার trick:

"FIT on TRAIN, TRANSFORM on TEST"

(শুধু Training data দেখে শেখো, Testing data-য় শুধু প্রয়োগ করো)

১ লাইনে সারসংক্ষেপ:

"Raw Data → Missing ভরাট → Text সংখ্যায় → সব একই scale-এ → Model!"

অথবা: "Clean করো, Encode করো, Scale করো — তারপর Train করো!"

Quick Reference Card

DATA PREPROCESSING CHEAT SHEET

MISSING VALUES

- Numerical + No Outlier → `fillna(mean)`
- Numerical + Outlier → `fillna(median)`
- Categorical → `fillna(mode)`
- > 70% missing → drop column
- Time Series → `ffill()` or `bfill()`

ENCODING

- Ordinal (ক্রম আছে) → `LabelEncoder()`
- Nominal (ক্রম নেই) → `pd.get_dummies(drop_first=True)`
- High Cardinality → Target Encoding

SCALING

- Neural Net, Image → `MinMaxScaler()` $[0,1]$
- SVM, PCA, LinReg → `StandardScaler()` $[\mu=0, \sigma=1]$
- Outlier sensitive → `StandardScaler (safer)`
- Tree-based models → No scaling needed! ✓

GOLDEN RULE: Always fit on TRAIN, transform on TEST!

📅 তৈরিৰ তাৰিখ: ২০২৬-০৪-০৪ | 🔗 পৰবৰ্তী নোট: *Linear Regression*